File Handling

Rizwan Rashid

Introduction

- Data stored in the program are temporary;
  - It get lost when the program terminates
- To permanently store the data created in a program,
  - Need to save them in a file on a disk or other permanent storage device
- The file can then be transported and read later by other programs

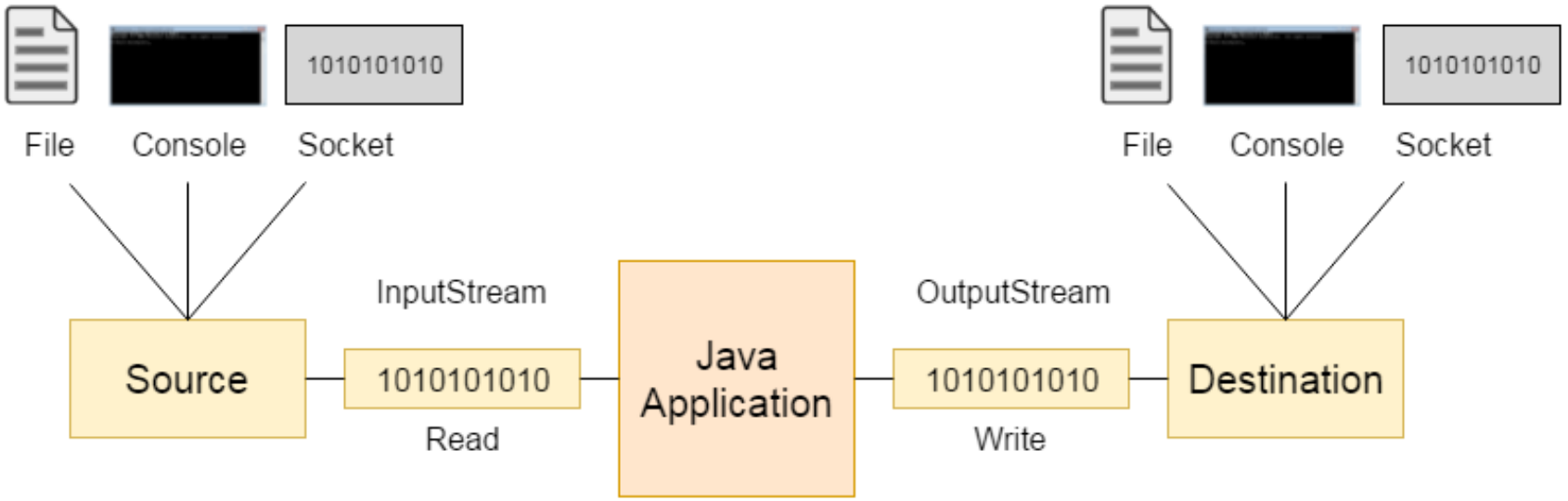
Introduction

- File Handling
  - Way to handle the files stored on the hard disk
  - So that we can, read and write the data in file
- Java provides **`java.io`** package
  - Contain all class that help in performing input and output operations on files

Java Streams

- Stream
  - A stream is a sequence of data.
  - In Java, a stream is composed of bytes called a stream
  - Stream represent Source(which generate the data in form of streams) and Destination (which consume or read data available on stream)

In `java.io` package all classes are divided into two type of streams
1- Character Stream
2- Byte Stream



Byte Stream (10011101011101011101)

- Raw sequence of 0's and 1’s to represent data
- The data belong to either audio, video or text.
- Just read/write data in form of 8bits(byte by byte)
- The classes that handle Byte Stream and which are descendent of
  - java.io.InputStream
  - java.io.OutputStream
  - java.io.RandomAccessFile

BufferedInputStream
BufferedoutputStream
ByteArrayInputStream
ByteArrayOutputStream
DataInputStream
DataOutputStream
FileInputStream
FileOutputStream
PrintStream

Character Stream (11110110 11010101 101010101)

- Character Stream is based on Byte stream, and bunch of 16 bits are picked as character
- Data must be characters
- Characters are encoded using a character encoding scheme such as Unicode
- The classes that handle Character Stream and which are descendent of
  - java.io.Reader
  - java.io.Writer class

BufferedReader
BufferedWriter
CharArrayReader
CharArrayWriter
FileReader
FileWriter
InputStreamReader
OutputStreamWriter
PrintWriter
StringReader
StringWriter

File Class

The **File** class contains the methods for obtaining the properties of a file/directory and for renaming and deleting a file/directory.

- Absolute file name(full name)
  - Contains a file name with its complete path and drive letter
  - Example "c:\book\Welcome.java"
- Relative file name (In relation to current directory
  - The complete directory path for a relative file name is omitted
  - Example: *Welcome.java* is a relative file name. If the current working directory is c:\book
- Do not use absolute file names in your program

java.io.File	
`+File(pathname: String)`	Creates a `File` object for the specified path name. The path name may be a directory or a file.
`+File(parent: String, child: String)`	Creates a `File` object for the child under the directory parent. The child may be a file name or a subdirectory.
`+File(parent: File, child: String)`	Creates a `File` object for the child under the directory parent. The parent is a `File` object. In the preceding constructor, the parent is a string.
`+exists(): boolean`	Returns true if the file or the directory represented by the `File` object exists.
`+canRead(): boolean`	Returns true if the file represented by the `File` object exists and can be read.
`+canWrite(): boolean`	Returns true if the file represented by the `File` object exists and can be written.
`+isDirectory(): boolean`	Returns true if the `File` object represents a directory.
`+isFile(): boolean`	Returns true if the `File` object represents a file.
`+isAbsolute(): boolean`	Returns true if the `File` object is created using an absolute path name.
`+isHidden(): boolean`	Returns true if the file represented in the `File` object is hidden. The exact definition of *hidden* is system-dependent. On Windows, you can mark a file hidden in the File Properties dialog box. On Unix systems, a file is hidden if its name begins with a period(.) character.
`+getAbsolutePath(): String`	Returns the complete absolute file or directory name represented by the `File` object.
`+getCanonicalPath(): String`	Returns the same as `getAbsolutePath()` except that it removes redundant names, such as "." and "..", from the path name, resolves symbolic links (on Unix), and converts drive letters to standard uppercase (on Windows).
`+getName(): String`	Returns the last name of the complete directory and file name represented by the `File` object. For example, new `File("c:\\book\\test.dat").getName()` returns `test.dat`.
`+getPath(): String`	Returns the complete directory and file name represented by the `File` object. For example, new `File("c:\\book\\test.dat").getPath()` returns `c:\book\test.dat`.
`+getParent(): String`	Returns the complete parent directory of the current directory or the file represented by the `File` object. For example, new `File("c:\\book\\test.dat").getParent()` returns `c:\book`.
`+lastModified(): long`	Returns the time that the file was last modified.
`+length(): long`	Returns the size of the file, or 0 if it does not exist or if it is a directory.
`+listFile(): File[]`	Returns the files under the directory for a directory `File` object.
`+delete(): boolean`	Deletes the file or directory represented by this `File` object.The method returns true if the deletion succeeds.
`+renameTo(dest: File): boolean`	Renames the file or directory represented by this `File` object to the specified name represented in dest. The method returns true if the operation succeeds.
`+mkdir(): boolean`	Creates a directory represented in this `File` object. Returns true if the the directory is created successfully.
`+mkdirs(): boolean`	Same as `mkdir()` except that it creates directory along with its parent directories if the parent directories do not exist.

File Input and Output

Use the `Scanner` class for **reading text data** from a file and the `PrintWriter` class for **writing text data** to a file.

- A File object encapsulates the properties of a file or a path
  - Does not contain the methods for writing/reading data to/from a file
  - Referred to as data input and output, or I/O
- To perform I/O
  - Create objects using appropriate Java I/O classes

Writing Data Using `PrintWriter`

- The **java.io.PrintWriter** class can be used to create a file and write data to a text file
- Can use **print**, **println**, and **printf** methods on the **PrintWriter** object to write data to a file

Writing Data Using PrintWriter

java.io.PrintWriter	
+PrintWriter(filename: String)	Creates a PrintWriter for the specified file.
+print(s: String): void	Writes a string.
+print(c: char): void	Writes a character.
+print(cArray: char[]): void	Writes an array of character.
+print(i: int): void	Writes an int value.
+print(l: long): void	Writes a long value.
+print(f: float): void	Writes a float value.
+print(d: double): void	Writes a double value.
+print(b: boolean): void	Writes a boolean value.
Also contains the overloaded println methods.	A println method acts like a print method; additionally it prints a line separator. The line separator string is defined by the system. It is \r\n on Windows and \n on Unix.
Also contains the overloaded printf methods.	

Try-with-resources

Programmers often forget to close the file. JDK 7 and above provides the followings new try-with-resources syntax that automatically closes the files.

try (declare and create resources) {

  Use the resource to process the file;

}

```
 try( PrintWriter output = new PrintWriter(file); )
 {
    output.print("John T Smith ");
    output.println(90);
    output.print("Eric K Jones ");
    output.println(85);
 }
```

Reading Data Using `Scanner`

- To read from keyboard
  - Scanner input = new Scanner(System.in);
- To read from file
  - Scanner input = new Scanner(new File(fileName));

Reading Data Using `Scanner`

java.util.Scanner	
+Scanner(source: File)	Creates a Scanner object to read data from the specified file.
+Scanner(source: String)	Creates a Scanner object to read data from the specified string.
+close()	Closes this scanner.
+hasNext(): boolean	Returns true if this scanner has another token in its input.
+next(): String	Returns next token as a string.
+nextByte(): byte	Returns next token as a byte.
+nextShort(): short	Returns next token as a short.
+nextInt(): int	Returns next token as an int.
+nextLong(): long	Returns next token as a long.
+nextFloat(): float	Returns next token as a float.
+nextDouble(): double	Returns next token as a double.
+useDelimiter(pattern: String): Scanner	Sets this scanner's delimiting pattern.

```
public static void main(String[] args){
                        try{
                                File file = new File("File/name1.txt");
                                Scanner input =  new Scanner(file);


                                while(input.hasNext()){
                                        String fName = input.next();
                                        String mName = input.next();
                                        String lName = input.next();
                                        int age      = input.nextInt();
                                        System.out.println(fName+" "+mName+"
"+lName+" "+age);
                                }
                                input.close();
                        }
                        catch(IOException e){

                        }

        }
```